RAILWAY BOOKING SYSTEM

TEAM WORK INSTRUCTION MANUAL

Project Goal

Build a complete railway ticket booking website with a modular frontend, backend APIs, and shared database.

Frontend: HTML, CSS, JavaScript, React
Backend: MERN-based architecture / Node.js + Express
Database: SQL schema defined in this document
Design Inspiration: IRCTC
Theme: Summer Ocean Breeze

---

1. COMMON RULES — ALL GROUPS

Every group MUST follow these rules.

UI / Design

- Follow the common **Summer Ocean Breeze** theme.
- Take layout inspiration from IRCTC, but do not copy it directly.
- Main heading: **22px**
- Sub-heading: **18px**
- Normal text: **12px**
- Maintain the same Navbar, typography, buttons, spacing and overall design across all pages.
- Every page must be responsive.
- Do not create independent designs that conflict with other groups.

GitHub

Each group works on its own branch.

Recommended branch format:

group1-train-search

group2-payment-ticket

group3-auth-user

group4-booking

group5-seat-management

Before pushing:

git pull

git add .

git commit -m "Meaningful commit message"

git push

Do NOT directly modify another group's module without discussion.

Integration Rule

Every group must use the **exact table names, column names, primary keys and foreign keys** specified below.

Do not independently rename:

user_id

train_id

station_id

schedule_id

seat_id

booking_id

passenger_id

payment_id

ticket_id

cancellation_id

refund_id

This is necessary for final database integration.

---

2. FINAL GROUP ALLOCATION

Group	Members	Module	Main Database
G1	Prathmesh, Asim, Aditya	Train Search & Details	stations, trains, routes, schedules

Group	Members	Module	Main Database
G2	Angad, Ashish, Sarthak	Payment & Ticket	payments, tickets, cancellations, refunds
G3	Sakshi, Saloni, Deepak	Authentication & User	users
G4	Prajkta, Vaishnavi, Shivani	Booking	bookings, passengers
G5	Subodh, Kshitij, Aniket	Seat Management	seats

This allocation is the working allocation for the project.

---

3. GROUP 1 — TRAIN SEARCH & DETAILS

Objective

Build everything required for the user to search for trains and view complete train information before booking.

Frontend Pages

Create:

1. Train Search Page

2. Train Results/List Page

3. Train Details Page

4. Train Schedule Page

Train Search Page

User should be able to enter:

From Station

To Station

Journey Date

Class

Example:

From: Pune

To: Mumbai

Date: 15/10/2026

Class: 3A

The page should send the search request to the backend.

Train Results Page

Display:

Train Number

Train Name

Train Type

Departure

Arrival

Duration

Available Classes

Provide:

View Details

Select Train / Continue Booking

Train Details Page

Display:

Train Number

Train Name

Train Type

Route

Stations

Arrival Time

Departure Time

Distance

Schedule

Schedule Page

Display the train's journey date and schedule.

G1 DATABASE TABLES

1. stations

station_id INT PK

station_code VARCHAR(10) UNIQUE

station_name VARCHAR(100)

city VARCHAR(100)

state VARCHAR(100)

2. trains

train_id INT PK

train_number VARCHAR(20) UNIQUE

train_name VARCHAR(100)

train_type VARCHAR(50)

total_coaches INT

3. routes

route_id INT PK

train_id INT FK → trains.train_id

station_id INT FK → stations.station_id

stop_number INT

arrival_time TIME

departure_time TIME

distance_km DECIMAL(8,2)

4. schedules

schedule_id INT PK

train_id INT FK → trains.train_id

journey_date DATE

status ENUM

Status:

SCHEDULED

CANCELLED

COMPLETED

G1 APIs

Create APIs for:

GET /stations

GET /stations/:id

GET /trains

GET /trains/:id

GET /trains/search

GET /trains/:id/routes

GET /trains/:id/schedule

G1 Output

By the end, G1 must provide:

✓ Search UI

✓ Train results UI

✓ Train details UI

✓ Schedule UI

✓ Station API

✓ Train API

✓ Route API

✓ Schedule API

✓ Database tables

✓ Sample data

✓ GitHub branch

---

4. GROUP 2 — PAYMENT & TICKET

Objective

Handle everything after booking regarding payment, ticket generation, cancellation and refund.

Frontend Pages

Create:

1. Payment Page

2. Payment Processing/Status Page

3. Booking Confirmation Page

4. Ticket Page

5. My Bookings Page

6. Cancellation Page

7. Refund Status Page

Payment Page

Display:

PNR

Passenger Details

Train Details

Journey Details

Total Amount

Payment methods:

UPI

CARD

NETBANKING

WALLET

Payment Status

Handle:

PENDING

SUCCESS

FAILED

REFUNDED

Do not show the ticket as confirmed before successful payment.

Ticket Page

Display:

Ticket Number

PNR

Passenger Name

Train Number

Train Name

Source

Destination

Journey Date

Seat / Coach

Booking Status

Provide:

Download / Print Ticket

Cancel Ticket

Cancellation

User should be able to:

View booking

Select cancellation

Confirm cancellation

See refund amount

G2 DATABASE TABLES

payments

payment_id INT PK

booking_id INT FK → bookings.booking_id

amount DECIMAL(10,2)

payment_method ENUM

transaction_id VARCHAR(100) UNIQUE

payment_status ENUM

payment_date TIMESTAMP

Payment methods:

UPI

CARD

NETBANKING

WALLET

Payment statuses:

PENDING

SUCCESS

FAILED

REFUNDED

---

tickets

ticket_id INT PK

booking_id INT FK → bookings.booking_id

ticket_number VARCHAR(30) UNIQUE

issued_at TIMESTAMP

ticket_status ENUM

Ticket status:

VALID

CANCELLED

---

cancellations

cancellation_id INT PK

booking_id INT FK → bookings.booking_id

cancelled_at TIMESTAMP

reason VARCHAR(255)

refund_amount DECIMAL(10,2)

---

refunds

refund_id INT PK

payment_id INT FK → payments.payment_id

booking_id INT FK → bookings.booking_id

refund_amount DECIMAL(10,2)

refund_status ENUM

refund_date TIMESTAMP

Refund status:

PENDING

PROCESSED

FAILED

G2 APIs

POST /payments

GET /payments/:booking_id

POST /tickets

GET /tickets/:booking_id

POST /cancellations

GET /cancellations/:booking_id

POST /refunds

GET /refunds/:booking_id

G2 Output

✓ Payment UI

✓ Payment status

✓ Ticket UI

✓ My Bookings UI

✓ Cancellation UI

✓ Refund UI

✓ Payment APIs

✓ Ticket APIs

✓ Cancellation APIs

✓ Refund APIs

✓ Database tables

✓ GitHub branch

---

5. GROUP 3 — AUTHENTICATION & USER

Objective

Build complete user authentication and profile management.

Frontend Pages

Create:

1. Login
2. Register
3. Forgot Password
4. Reset Password
5. User Profile
6. Account Dashboard

Register

Fields:

Name

Email

Phone

Password

Confirm Password

Validation must be implemented.

Login

User enters:

Email

Password

On successful authentication:

Generate JWT

Store authentication state

Redirect to dashboard/home

Profile

Display:

Name

Email

Phone

Role

Account creation date

Allow appropriate profile updates.

Authentication Requirements

Implement:

Password hashing

JWT authentication

Protected routes

Login validation

Registration validation

Logout

Never store plain-text passwords.

G3 DATABASE TABLE

users

user_id INT PK

name VARCHAR(100) NOT NULL

email VARCHAR(100) UNIQUE NOT NULL

phone VARCHAR(15) UNIQUE

password VARCHAR(255) NOT NULL

role ENUM DEFAULT 'USER'

created_at TIMESTAMP

Role:

USER

ADMIN

G3 APIs

POST /auth/register

POST /auth/login

POST /auth/logout

POST /auth/forgot-password

POST /auth/reset-password

GET /users/profile

PUT /users/profile

Protected APIs must require valid authentication.

G3 Output

✓ Register

✓ Login

✓ Forgot password

✓ Reset password

✓ Profile

✓ JWT authentication

✓ Password hashing

✓ Protected routes

✓ User API

✓ users table

✓ GitHub branch

---

6. GROUP 4 — BOOKING

Objective

Handle the actual railway booking process after the user selects a train.

The booking flow should be:

Train Search

↓

Train Details

↓

Select Seat

↓

Enter Passenger Details

↓

Booking Summary

↓

Payment

Frontend Pages

Create:

1. Passenger Details

2. Booking Details

3. Booking Summary

4. Booking Confirmation

Passenger Details

For each passenger:

Name

Age

Gender

Seat Preference

Allow multiple passengers in one booking.

Booking Summary

Display:

Train

Train Number

Source

Destination

Journey Date

Passengers

Selected Seats

Fare

Total Amount

Provide:

Proceed to Payment

G4 DATABASE TABLES

bookings

booking_id INT PK

user_id INT FK → users.user_id

schedule_id INT FK → schedules.schedule_id

source_station_id INT FK → stations.station_id

destination_station_id INT FK → stations.station_id

booking_date TIMESTAMP

total_amount DECIMAL(10,2)

status ENUM

pnr VARCHAR(20) UNIQUE

Booking status:

PENDING

CONFIRMED

CANCELLED

---

passengers

passenger_id INT PK

booking_id INT FK → bookings.booking_id

name VARCHAR(100)

age INT

gender ENUM

seat_preference VARCHAR(30)

Gender:

MALE

FEMALE

OTHER

G4 APIs

POST /bookings

GET /bookings/:id

GET /bookings/user/:user_id

PUT /bookings/:id

POST /bookings/:id/confirm

Passenger APIs:

POST /passengers

GET /passengers/:booking_id

PUT /passengers/:id

DELETE /passengers/:id

G4 Output

✓ Passenger page

✓ Booking page

✓ Booking summary

✓ Booking confirmation

✓ Booking API

✓ Passenger API

✓ bookings table

✓ passengers table

✓ PNR generation

✓ GitHub branch

---

7. GROUP 5 — SEAT MANAGEMENT

Objective

Manage train seats and provide the seat-selection interface.

G5 works closely with **G4 Booking**.

Frontend Pages

Create:

1. Seat Selection Page

2. Coach View

3. Seat Availability

4. Booking Seat Summary

Seat Selection

Display:

Coach

Seat Number

Class

Availability

Seat states:

AVAILABLE

SELECTED

BOOKED

BLOCKED

The user should not be able to select:

BOOKED

BLOCKED

seats.

G5 DATABASE TABLE

seats

seat_id INT PK

train_id INT FK → trains.train_id

coach_number VARCHAR(10)

seat_number VARCHAR(10)

class_type ENUM

status ENUM

Class types:

SL

3A

2A

1A

CC

2S

Seat status:

AVAILABLE

BOOKED

BLOCKED

G5 APIs

GET /seats/:train_id

GET /seats/:train_id/available

GET /seats/:train_id/:class_type

POST /seats/:seat_id/block

POST /seats/:seat_id/book

POST /seats/:seat_id/release

IMPORTANT G5 + G4 INTEGRATION

G5 owns the **seat availability and seat management logic**.

G4 owns the **booking and passenger information**.

Therefore:

G5

Seat Selection

↓

Selected seat IDs

↓

G4

Booking

↓

Passenger + Booking

↓

G2

Payment

↓

Ticket

Do not duplicate seat-management logic inside G4.

G5 Output

✓ Seat selection UI

✓ Coach UI

✓ Seat availability

✓ Seat API

✓ Seat booking/release logic

✓ seats table

✓ Sample seat data

✓ GitHub branch

---

8. COMPLETE DATABASE OWNERSHIP

This is the final database division.

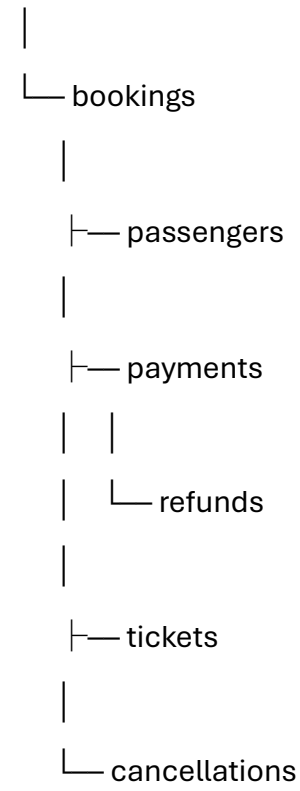
Group	Tables	Primary Responsibility
G1	stations	Station information
G1	trains	Train information
G1	routes	Train routes
G1	schedules	Journey schedules
G2	payments	Payment processing
G2	tickets	Ticket generation
G2	cancellations	Cancellation records
G2	refunds	Refund records
G3	users	Authentication & users
G4	bookings	Booking records
G4	passengers	Passenger information
G5	seats	Seat availability & management

---

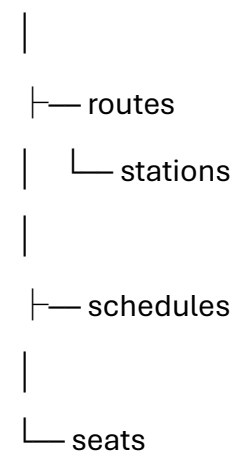
9. MASTER FOREIGN-KEY RELATIONSHIP

The database integration must follow this structure:

users



trains



Additional relationships:

bookings.user_id

↓

users.user_id

bookings.schedule_id

↓

schedules.schedule_id

bookings.source_station_id

↓

stations.station_id

bookings.destination_station_id

↓

stations.station_id

passengers.booking_id

↓

bookings.booking_id

payments.booking_id

↓

bookings.booking_id

tickets.booking_id

↓

bookings.booking_id

cancellations.booking_id

↓

bookings.booking_id

refunds.booking_id

↓

bookings.booking_id

refunds.payment_id

↓

payments.payment_id

seats.train_id

↓

trains.train_id

routes.train_id

↓

trains.train_id

routes.station_id

↓

stations.station_id

schedules.train_id

↓

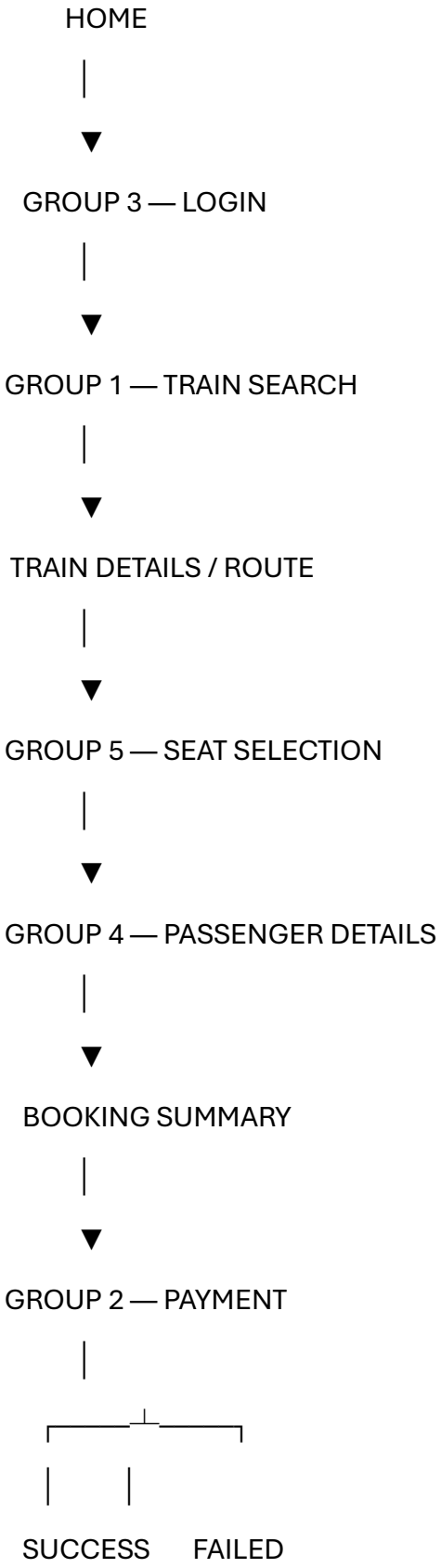
trains.train_id

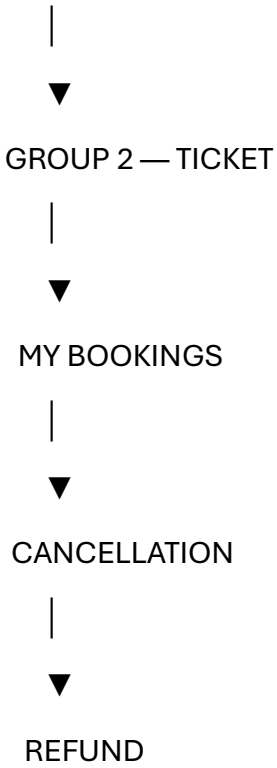
These relationships are based on the database schema supplied in the project document.

---

10. COMPLETE USER FLOW

Every group must understand where its module fits.





---

11. INTEGRATION CONTRACT

Before merging, every group MUST provide:

1. Frontend pages

2. Backend routes

3. Controllers

4. Database queries/models

5. API documentation

6. Sample request

7. Sample response

8. Database sample data

9. GitHub branch

10. README/update documentation

Each API should document:

METHOD

ENDPOINT

REQUEST BODY

RESPONSE

ERROR RESPONSE

REQUIRED AUTHENTICATION

Example:

POST /bookings

Request:

```
{
  "user_id": 1,
  "schedule_id": 5,
  "source_station_id": 10,
  "destination_station_id": 20
}
```

Response:

```
{
  "booking_id": 101,
  "pnr": "PNR123456",
  "status": "PENDING"
}
```

---

12. DO NOT DO THESE THINGS

Do NOT

❌ Rename database columns

❌ Create duplicate tables

❌ Create different versions of the same API

❌ Hardcode another group's database logic

❌ Change primary keys

❌ Change foreign-key relationships

❌ Push directly to main

❌ Commit unfinished/broken code

❌ Use different UI themes

❌ Change API response structure without informing the concerned group

DO

✓ Follow the shared schema

✓ Follow common UI

✓ Use Git branches

✓ Pull before starting work

✓ Commit regularly

✓ Test APIs

✓ Share dependencies with other groups

✓ Document your APIs

✓ Provide sample data

✓ Test integration before merge

---

13. GROUP-TO-GROUP DEPENDENCIES

Group	Depends On	Provides To
G1 Train Search	None	G4, G5
G2 Payment/Ticket	G4	User
G3 Authentication	None	G4, G2, all protected pages
G4 Booking	G1, G3, G5	G2
G5 Seat Management	G1	G4

Most Important Dependencies

G1 → G5

G1 → G4

G3 → G4

G5 → G4

G4 → G2

Therefore, communication between these groups is mandatory.

---

14. WHAT EACH GROUP MUST COMPLETE

G1

Train Search

Train List

Train Details

Schedule

Station APIs

Train APIs

Route APIs

Schedule APIs

4 database tables

Sample data

G2

Payment

Payment Status

Ticket

My Bookings

Cancellation

Refund

4 database tables

Payment/Ticket APIs

G3

Register

Login

Logout

Forgot Password

Reset Password

Profile

JWT

Password Hashing

Protected Routes

users table

Authentication APIs

G4

Passenger Details

Booking

Booking Summary

Booking Confirmation

PNR

bookings table

passengers table

Booking APIs

Passenger APIs

G5

Seat Selection

Coach View

Seat Availability

Seat Blocking

Seat Booking

Seat Release

seats table

Seat APIs

---

15. FINAL DEFINITION OF DONE

A group is considered **DONE** only when all of the following are completed:

[ ] Frontend pages completed

[ ] Responsive UI completed

[ ] Backend APIs completed

[ ] Database tables completed

[ ] Foreign keys verified

[ ] Sample data inserted

[ ] APIs tested

[ ] Frontend connected to APIs

[ ] Error handling implemented

[ ] Git branch updated

[ ] Code committed

[ ] README/documentation updated

[ ] Other dependent groups informed

[ ] Integration tested

---

16. TEAM MEETING — QUESTIONS EVERY GROUP MUST ANSWER

At every progress meeting, each group must answer:

WHO?

Who is responsible for each page?

Who is responsible for backend?

Who is responsible for database?

WHAT?

What exactly are you implementing?

What APIs are required?

What tables are being used?

BY WHEN?

When will frontend be completed?

When will backend be completed?

When will integration be completed?

EXPECTED OUTPUT?

What pages will be delivered?

What APIs will be delivered?

What database tables will be delivered?

What sample data will be delivered?

DEPENDENCY?

Which other group do you depend on?

Which group depends on your module?

FINAL TEAM RULE

Each group owns its module, but the project has ONE common database and ONE common application.

Therefore:

BUILD SEPARATELY

↓

FOLLOW SAME SCHEMA

↓

FOLLOW SAME API CONTRACT

↓

TEST INDIVIDUALLY

↓

INTEGRATE

↓

TEST COMPLETE SYSTEM

Do not optimize only for your own page. Build your module so that it can be integrated directly with the other four groups.